

Florence dashboard, January 1420

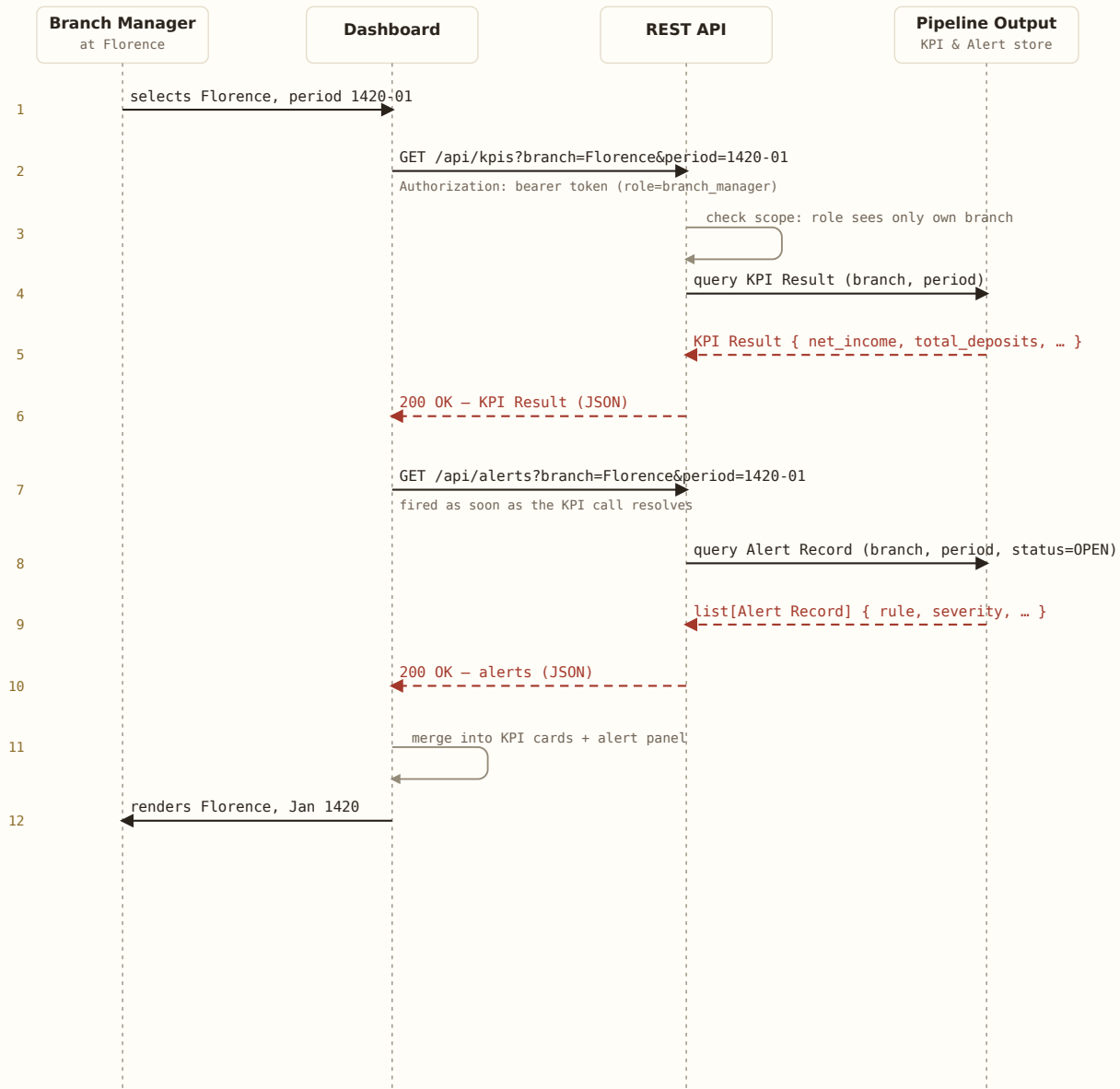
One real request, traced end to end — a branch manager opening the Florence branch dashboard for January 1420 — so the three modules (dashboard, API, pipeline output) provably plug together before anyone writes code against them.

What a sequence diagram is for

A **sequence diagram** shows one specific scenario over time, not the whole system. Each vertical line below is a “lifeline” — one participant, alive for the length of the story — and each horizontal arrow is one message between them, read top to bottom in the order it actually happens. It's the difference between a subway map (every possible route) and a single rider's trip receipt (exactly what happened, in order). We're drawing the receipt.

Why this matters for us: the 1-week plan calls for exactly this diagram before Matt builds the API or Sloane's KPI output format is locked in. If the dashboard expects a field the API never sends, this is where that gap shows up — on paper, for free — instead of during Day 5 integration testing.

The trace



Solid ink arrows are requests moving down the chain; dashed accent arrows carry a data-contract payload back up (**KPI Result** or **Alert Record**); faint loops are internal work a participant does on its own lifeline.

-
- request / instruction - - - response carrying a data contract
 - internal step, no message sent

Step by step

- 1 **Branch Manager → Dashboard.** Picks Florence and January 1420 from the branch dashboard's period selector. Nothing has left the browser yet.
- 2 **Dashboard → API.** Fires `GET /api/kpis` with `branch` and `period` as query parameters, and the manager's auth token in the header.
- 3 **API, internally.** Reads the role out of the token and enforces the access-control rule from Section 3.2 of the plan: a branch manager's token is only ever allowed to resolve to their own branch, no matter what `branch` value is in the query string.
- 4 **API → Pipeline Output.** Looks up the pre-computed **KPI Result** record for `(Florence, 1420-01)` — the API never recomputes KPIs live, it reads what Sloane's Phase 3 module already produced.
- 5 **Pipeline Output → API.** Returns that one KPI Result record: cash position, deposits, loan portfolio, expenses, revenue, net income — the exact fields on the Data-Contract Reference Sheet.
- 6 **API → Dashboard.** Serializes it to JSON and responds `200 OK`.
- 7 **Dashboard → API.** A second call, `GET /api/alerts`, same branch and period — the alert panel is a separate contract from the KPI cards, so it's a separate request.
- 8 **API → Pipeline Output.** Looks up any **Alert Records** Sloane's Phase 4 anomaly module raised for that branch and period, filtered to `status = OPEN`.
- 9 **Pipeline Output → API.** Returns the matching alert list, each one carrying the rule that fired (A-G), its severity, and the transaction IDs it flagged.
- 10 **API → Dashboard.** Responds `200 OK` with the alert list.
- 11 **Dashboard, internally.** Merges both responses into the screen: KPI cards from the first payload, the alert panel from the second.
- 12 **Dashboard → Branch Manager.** The manager sees Florence's January 1420 numbers and any open alerts — the whole round trip, on the diagram, in two API calls.

The two contracts doing the work

Neither payload above is invented by the API — both are the shared data contracts from diagram 4, produced upstream by Sloane's modules and only *read* by the API layer.

KPI RESULT One record per (branch, period) : cash position, deposits/withdrawals, loan portfolio, operating expenses, revenue & net income. Produced by Phase 3.	ALERT RECORD One record per rule firing: which rule (A-G), severity, affected transaction IDs, status. Produced by Phase 4, from the same KPI-period slice.
---	--

Diagram 6 of 8 from the 1-Week Project Plan · MediciMessDelta · traces the scenario named in Section 4: a branch manager opening the Florence dashboard for January 1420.